



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4306 · Design & Analysis of Algorithms · Fall 2026

Workshop I — Analysis

No new material. Six stations, worked at the board and the keyboard, on Sessions 1–6

Session 7 of 24 · Friday 9 October 2026 · 8:30 – 9:50 AM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 7 OF 24

Workshop I — Analysis

Friday, 9 October 2026 · 8:30 – 9:50 AM

WHAT THIS SESSION IS FOR

- ✓ Turn recognition into fluency: everything here has already been taught.
- ✓ Surface the misunderstandings that only appear when a student has to produce an answer alone.
- ✓ Give the class time to be *stuck*, which is where learning happens and which lecture never allows.
- ✓ Fix the analysis vocabulary before recursion arrives on Tuesday and starts using it.

How to run today. Six stations, roughly twelve minutes each. Students work in pairs and write on the board; the instructor circulates rather than lectures. Solutions are printed here for the instructor and are distributed to students **after** the session, not before. If the class stalls on a station, that station is the lecture — abandon the rest of the plan and stay there.

THE SIX IDEAS BEING CONSOLIDATED

1. An algorithm is the idea; a program is one incarnation. 2. Cost is a function of input size; read tables sideways. 3. Drop constants, keep the dominant term, think large n . 4. O is a ceiling, Ω a floor, Θ the exact shape; proofs need a witness. 5. Sequences add, nests multiply, dependent nests need the series. 6. Case and bound are independent; worst case is the default.

Station 1 • Classify by Doubling

12 min

Each row reports measured times. No code is given. Identify the growth class from the numbers alone, then predict the missing entry.

Program	n = 1000	n = 2000	n = 4000	n = 8000	class?
P	0.8 s	1.6 s	3.2 s	?	
Q	0.5 s	2.0 s	8.0 s	?	
R	1.2 s	1.3 s	1.4 s	?	
S	1.0 s	2.2 s	4.8 s	?	
T	3.0 s	3.0 s	3.0 s	?	

SOLUTIONS

P doubles $\rightarrow \Theta(n) \rightarrow 6.4$ s. **Q** quadruples $\rightarrow \Theta(n^2) \rightarrow 32$ s. **R** grows by a constant additive amount per doubling $\rightarrow \Theta(\log n) \rightarrow$ about 1.5 s. **S** slightly more than doubles $\rightarrow \Theta(n \log n) \rightarrow$ about 10.4 s. **T** unchanged $\rightarrow \Theta(1) \rightarrow 3.0$ s.

Discussion point: S is the one students get wrong, calling it linear. Push them to compute the ratios: 2.2, 2.18, 2.18 — consistently above 2, and the excess is the log factor.

Station 2 • Count the Nest

12 min

PY GIVE Θ FOR EACH, AND THE EXACT COUNT WHERE YOU CAN

```
def f1(n):
    c = 0
    for i in range(n):
        for j in range(100):
            c += 1
    return c

def f2(n):
    c = 0
    for i in range(n):
        for j in range(i, n):
            c += 1
    return c
```

```
def f3(n):
    c = 0
    i = n
    while i > 1:
        for j in range(n):
            c += 1
        i //= 2
    return c

def f4(n):
    c = 0
    i = n
    while i > 1:
        for j in range(i):
            c += 1
        i //= 2
    return c

def f5(A):
    out = []
    for v in A:
        out.insert(0, v)
    return out
```

SOLUTIONS

f1: $100n \rightarrow \Theta(n)$. Nesting is not quadratic; the inner bound is constant.

f2: $n + (n-1) + \dots + 1 = n(n+1)/2 \rightarrow \Theta(n^2)$.

f3: outer runs $\log_2 n$ times, inner always $n \rightarrow \Theta(n \log n)$.

f4: inner runs $n, n/2, n/4, \dots$ summing to under $2n \rightarrow \Theta(n)$. Two nested loops, both depending on n , and the answer is linear.

f5: each `insert(0, v)` shifts everything $\rightarrow \Theta(n^2)$.

Discussion point: f3 and f4 differ by two characters and by a whole class. This pair is the best argument in the course for counting rather than pattern matching.

Station 3 • True or False, with a Reason

12 min

Claims

- (a) $3n^2 + 100n$ is $O(n^2)$
- (b) n is $O(n^2)$
- (c) n^2 is $O(n)$
- (d) n^2 is $\Omega(n \log n)$
- (e) 2^{n+1} is $\Theta(2^n)$
- (f) 2^{2n} is $\Theta(2^n)$

Solutions

- (a) **True** — Moves 1 and 2.
- (b) **True** — O is a ceiling, and it need not be tight.
- (c) **False** — no constant c makes $n^2 \leq cn$ for all large n .
- (d) **True** — n^2 grows at least as fast.

(g) $\log_2 n$ is $\Theta(\log_{10} n)$

(h) if f is $O(g)$ and g is $O(h)$ then f is $O(h)$

(e) **True** — $2^{n+1} = 2 \cdot 2^n$, a constant factor.

(f) **False** — $2^{2n} = (2^n)^2$, a squaring, not a constant factor.

(g) **True** — change of base multiplies by a constant.

(h) **True** — transitivity; compose the two witnesses.

Watch for this

(e) and (f) side by side is the point of the station. Students who answered (e) correctly by rule and (f) incorrectly by analogy have not yet internalised that the constant must multiply the *whole function*, not sit in the exponent.

Station 4 • Prove It From the Definition 12 min

Each student produces a witness pair (c, n_0) and the inequality that justifies it. Any valid witness is accepted; there is no best one.

- 1 Show that $7n + 40$ is $O(n)$.
- 2 Show that $n^2 + 5n + 6$ is $O(n^2)$.
- 3 Show that n^2 is **not** $O(n)$. (A proof by contradiction; suppose a witness existed.)
- 4 Show that $n^2 + n$ is $\Theta(n^2)$ — which requires *two* inequalities, not one.

SOLUTIONS

1. For $n \geq 40$, $40 \leq n$, so $7n + 40 \leq 8n$. Witness $c = 8$, $n_0 = 40$.

2. For $n \geq 6$, both $5n \leq 5n^2$ and $6 \leq n^2$, so the sum is $\leq 7n^2$. Witness $c = 7$, $n_0 = 6$.

3. Suppose $n^2 \leq cn$ for all $n \geq n_0$. Dividing by n gives $n \leq c$ for all large n , which fails as soon as $n > c$. No witness exists. $\square$

4. Upper: $n^2 + n \leq 2n^2$ for $n \geq 1$. Lower: $n^2 + n \geq n^2$ for $n \geq 0$. Witnesses $c_1 = 1$, $c_2 = 2$, $n_0 = 1$.

Discussion point: item 3 is the only one requiring a different *shape* of argument. Make sure every pair gets to it; it is the one that shows the definition has teeth.

Station 5 • At the Keyboard 12 min

Students bring their Session 6 homework code. Working in pairs, they run this experiment and write one paragraph of interpretation.

PY THE EXPERIMENT

```
import time, random

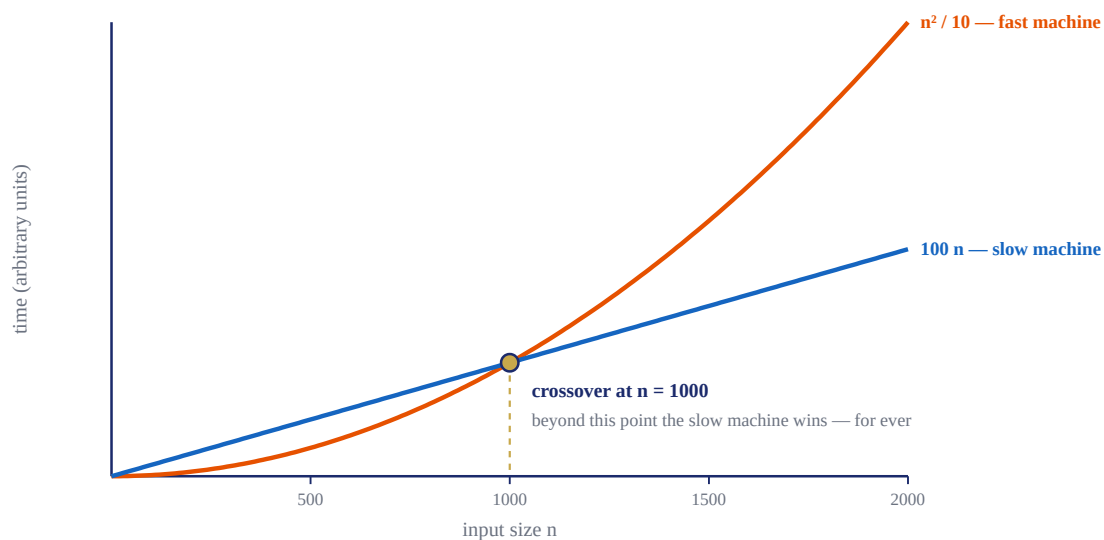
def timed(fn, *args, repeats=1):
    start = time.perf_counter()
    for _ in range(repeats):
        fn(*args)
    return (time.perf_counter() - start) * 1000 / repeats

for n in [1_000, 2_000, 4_000, 8_000]:
    A = random.sample(range(10 * n), n)
    T = -1 # guarantees no pair exists: the worst case
    q = timed(two_sum_quadratic, A, T)
    h = timed(two_sum_hash, A, T, repeats=50)
    print(n, round(q, 1), round(h, 3), round(q / h))
```

WHAT THEY SHOULD FIND, AND WHAT TO ASK THEM

The quadratic column multiplies by about four per doubling; the dictionary column by about two. The ratio column **grows**: roughly 190, 360, 700, 1240.

Ask: “If I gave you a computer ten times faster and left the brute-force code alone, what happens to the last column?” Answer: every entry is divided by ten, and it still grows. That is the Session 3 crossover argument, rediscovered by the students from their own data.



Worth re-drawing at the board while they have their own numbers in front of them. A faster machine buys a crossover point, never a victory. Their measured ratio column is this picture, sampled at four values of n .

Station 6 • Diagnose the Slow Code 12 min

Each snippet is real code of the kind students write. Find the accidental cost, state the class before and after, and fix it.

PY FOUR SNIPPETS TO REPAIR

```
# (a) build a list of the squares of the first n integers, in order
result = []
for i in range(n):
    result.insert(0, i * i)
result.reverse()

# (b) count how many elements of A also appear in B
count = 0
for v in A:
    if v in B:
        count += 1

# (c) find the largest gap between consecutive values in an unsorted list
best = 0
for i in range(len(A)):
    for j in range(len(A)):
        if A[i] > A[j]:
            best = max(best, A[i] - A[j])

# (d) remove duplicates, preserving order
out = []
for v in A:
    if v not in out:
        out.append(v)
```

SOLUTIONS

(a) $\Theta(n^2) \rightarrow \Theta(n)$. Use `append` and drop the `reverse` entirely; the code was reversing to undo damage it caused itself.

(b) $\Theta(n \cdot m) \rightarrow \Theta(n + m)$. Convert B to a set once, outside the loop.

(c) $\Theta(n^2) \rightarrow \Theta(n)$. The answer is simply `max(A) - min(A)`; the nest was computing one number the hard way. Note also that the stated goal (“consecutive values”) does not match what the code computes — a second, more serious bug.

(d) $\Theta(n^2) \rightarrow \Theta(n)$. Keep a `seen` set alongside the output list; `v not in out` is a linear scan every

time.

Discussion point: in (b) and (d) the fix is the same idea — *index it instead of scanning it* — and it is the same idea that took Two-Sum from $\Theta(n^2)$ to $\Theta(n)$.

WORKSHOP I TAKEAWAY

Six sessions in, the working toolkit is small: count the work, name its shape, say which inputs you mean, and check the claim against a clock. Everything after today — recursion, sorting, graphs, dynamic programming — is new *algorithms* analysed with these same four moves.

HOMEWORK · DUE TUESDAY 13 OCTOBER

1. Complete any station your pair did not finish. Solutions will be distributed after the session — attempt each one before reading them.
2. Write a one-page “cheat sheet” for Sessions 1–6, in your own words, on a single side of paper. You may bring it to the midterm. Writing it is the exercise; the permission is the incentive.
3. Take any program you have written for another course. Identify its most expensive loop, state the class, and say whether it could matter. Half a page.
4. Reading: CLRS §3.1–3.2 in full, now that you have used all of it.

Next session · Tuesday 13 October: Recursion. The loops vanish, and with them every counting rule we have just practised — a three-line function that will take longer than the age of your patience, and the new instrument that explains why.